Now that we know how d-way heaps work, the next question we need to ask is: wouldn't we be just fine with a regular, binary heap? Is there an advantage in a higher branching factor?

2.9.1 Do we need D-ary heaps?

Usually binary heaps are enough for all our programming needs.: the main advantage of this data structure is that it guarantees a logarithmic running time for each one of the common operations. In particular, being a binary balanced tree, the main operations are guaranteed to require a number of comparisons proportional, in the worst case, to $\log_2(N)$. As we discuss in appendix B, this guarantees us that we can run these methods on much larger containers than if the running time was linear: just consider that even with a billion elements, $\log_2(N)$ just evaluates to about 30.

As we have seen in the introduction, constant factors are irrelevant for the running time - i.e. $O(c*N) = O(N)$ - and we know from algebra that two logarithms with different bases only differs by a constant factor, in particular:

$$\log_b(N) = \log_2(N) / \log_2(b)$$

So, in conclusion, we have:

$$O(\log_2(N)) = O(\log_3(N)) = O(\log(N))$$